Context Management

Context is Claude's working memory. Every file it reads, every command it runs, every message you send - it all takes up space in the context window.

The context window · Compaction · Commands · Saving space

IN THIS DEMO, WE WILL

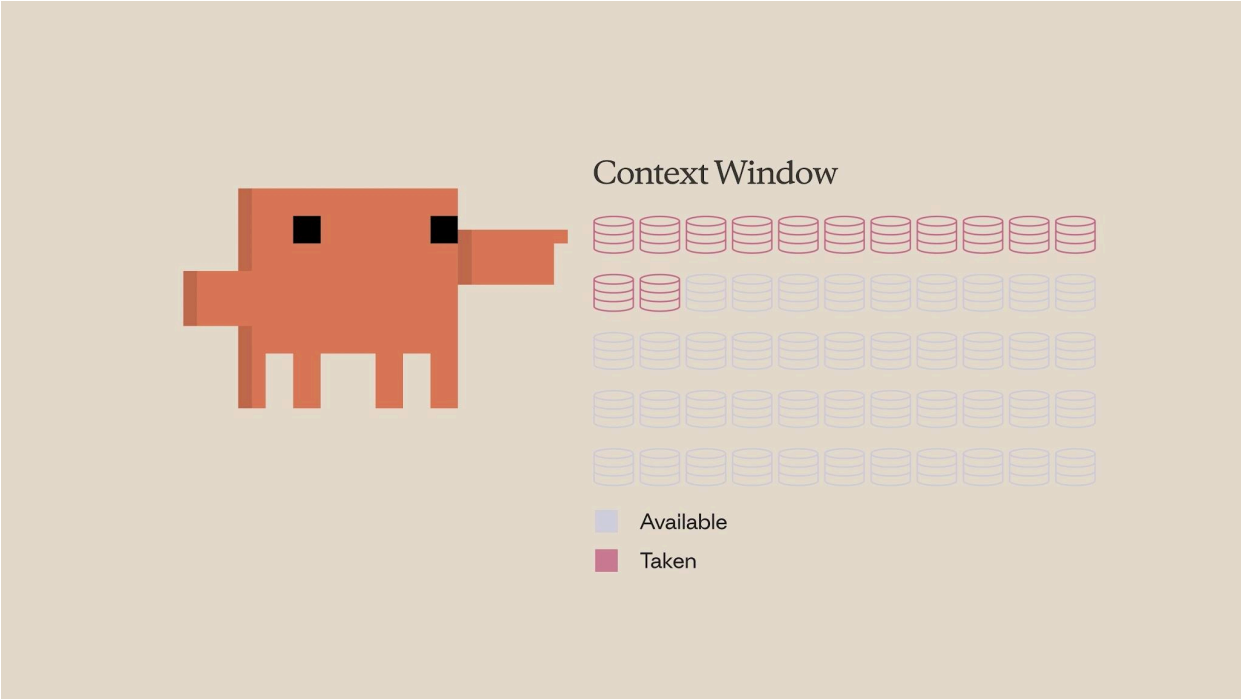
Learn how to manage context in Claude Code, covering:

- **What the context window is** - and why it's a finite resource
- What happens when it fills up: **automatic compaction**, and the summary it leaves behind
- The core commands - `/compact`, `/clear`, and `/context`
- When to reach for `/compact` vs. `/clear`, and how `CLAUDE.md` preserves memory across sessions
- Practical ways to **save context space** - be specific, manage MCP servers, and use subagents

ON THIS PAGE

01 What is the Context Window?

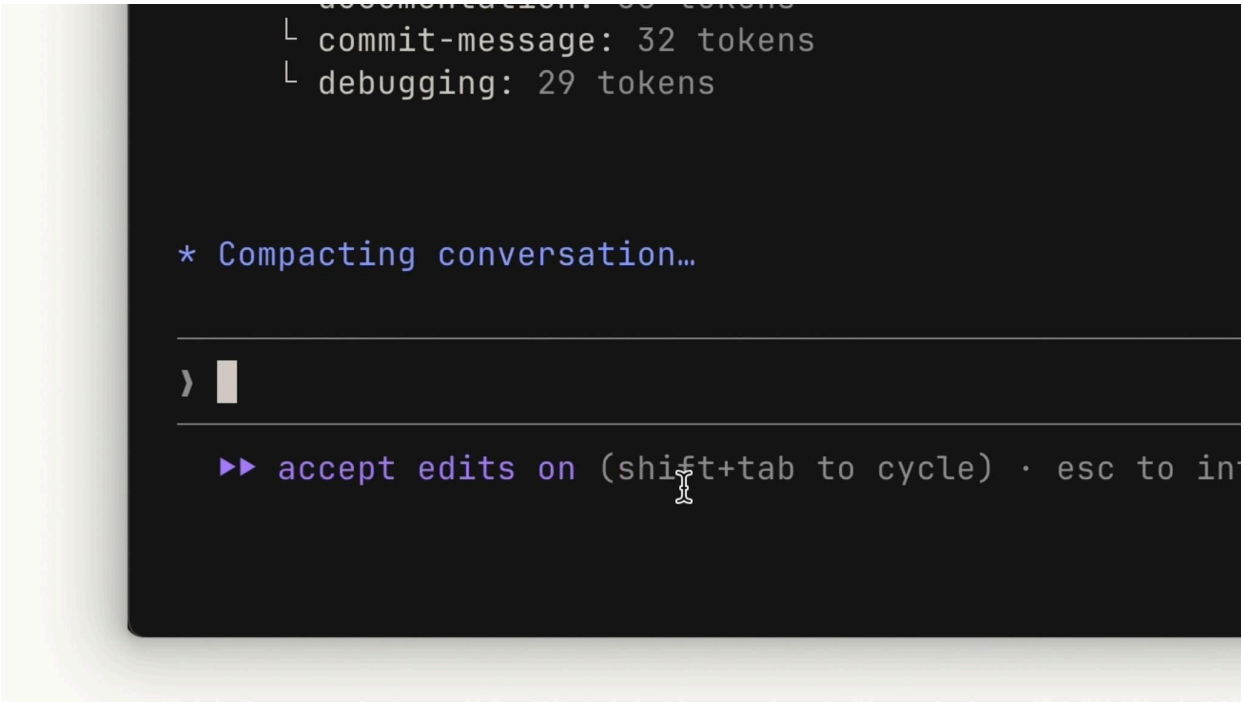
Think of the context window as the amount of space Claude can hold in its memory. Whenever you enter a prompt, Claude reads a file, runs a tool call, or receives a tool-call result, it all adds to the context window. Because that space is finite, it becomes important to optimize how you use it.



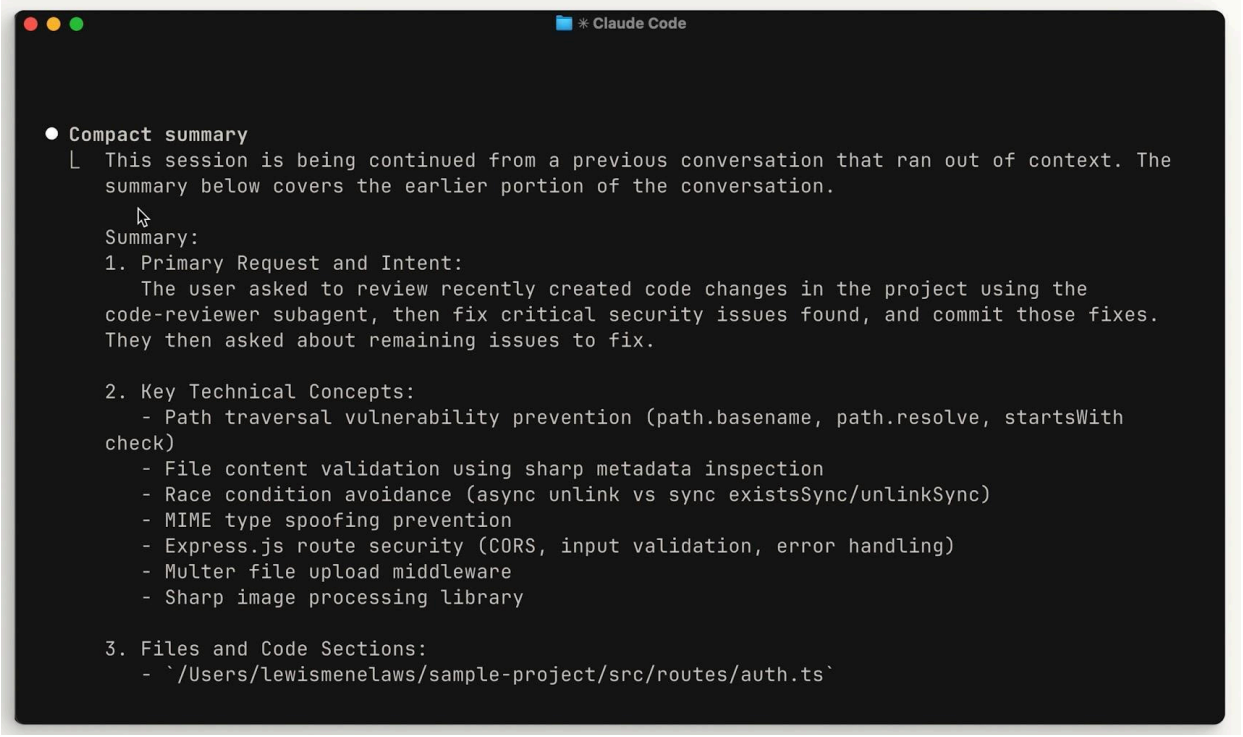
The context window is finite - every prompt, file, and tool result takes up a slice of it.

02 What Happens When Context Fills Up

When you approach the limit, the context window is automatically compacted. Compaction summarizes the important details and removes unnecessary tool-call results to free up space. Keep in mind that this process can lose some details along the way.



As you near the limit, Claude automatically compacts the conversation to free up space.



After compacting, a structured summary of the earlier conversation takes its place.

COMPACTION CAN LOSE DETAIL

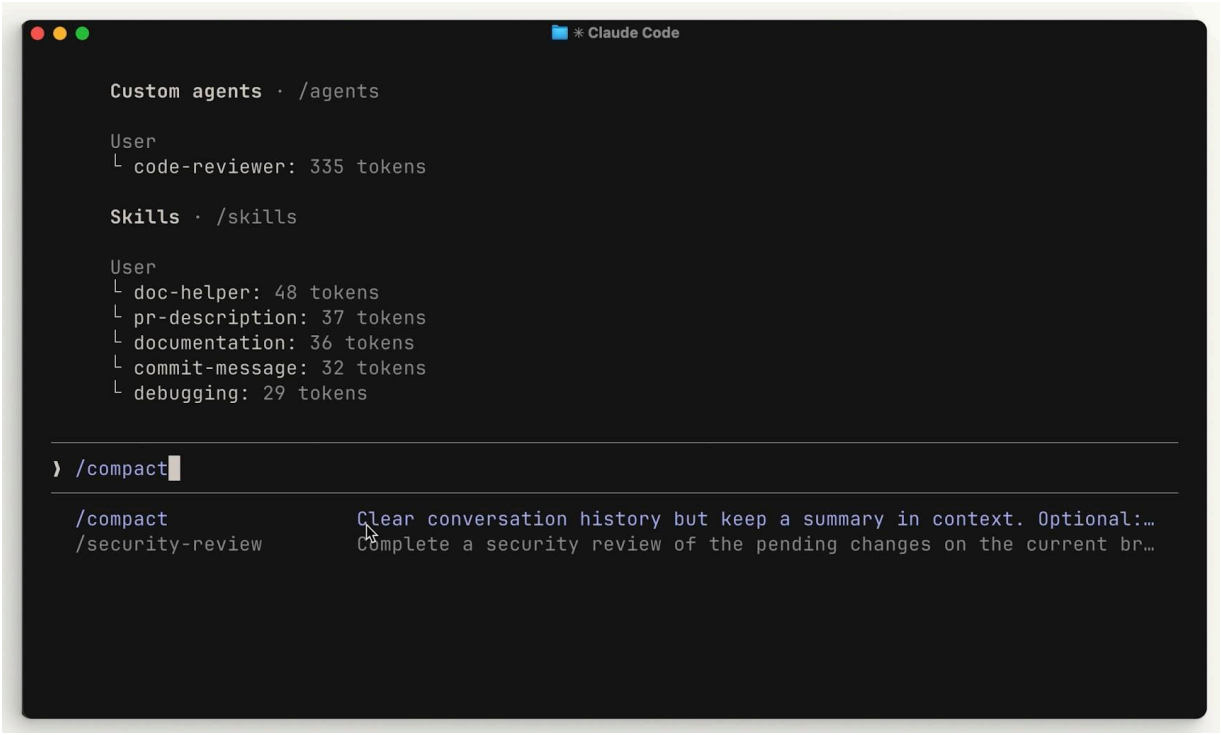
If something matters for the long term, don't rely on it surviving compaction - save it somewhere durable, such as your `CLAUDE.md` file.

03 Commands

Claude Code gives you three commands for managing context directly.

`/compact` - keep a summary

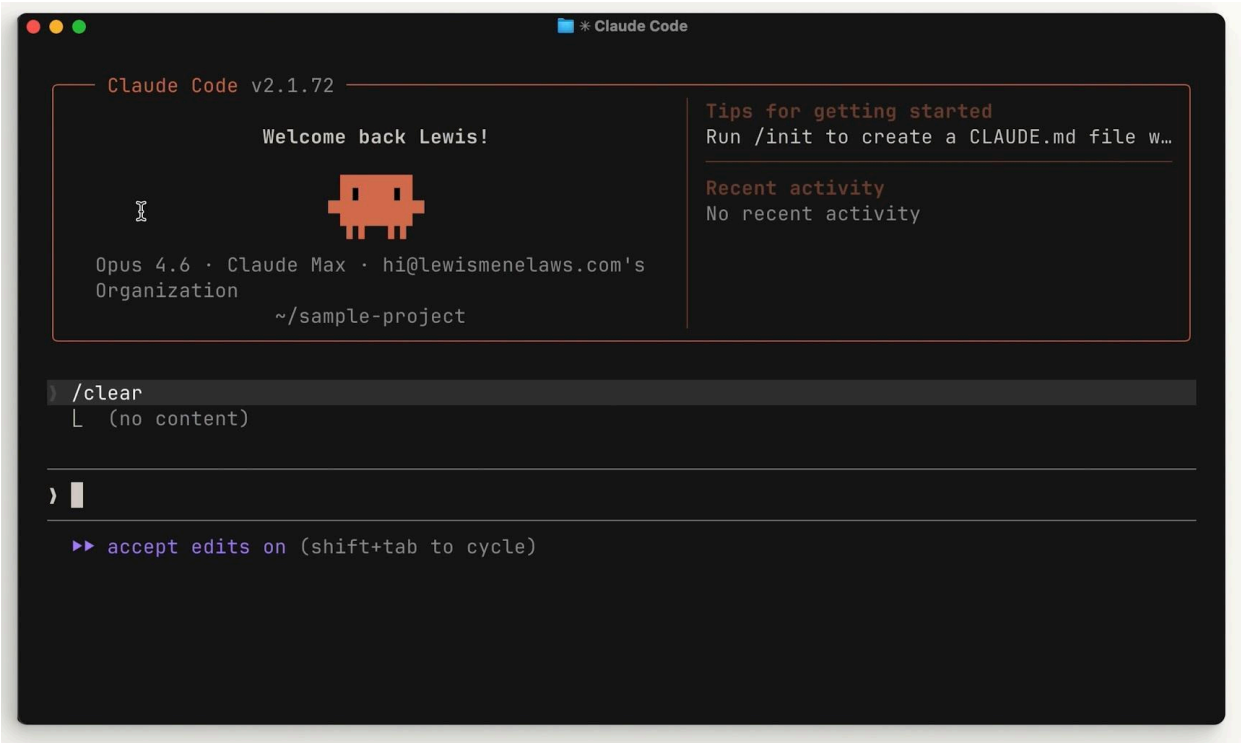
Run `/compact` to compact everything up to that point. It's handy when you want to free up context space while keeping a memory of what you previously worked on.



/compact clears the conversation history but keeps a summary in context.

`/clear` - start fresh

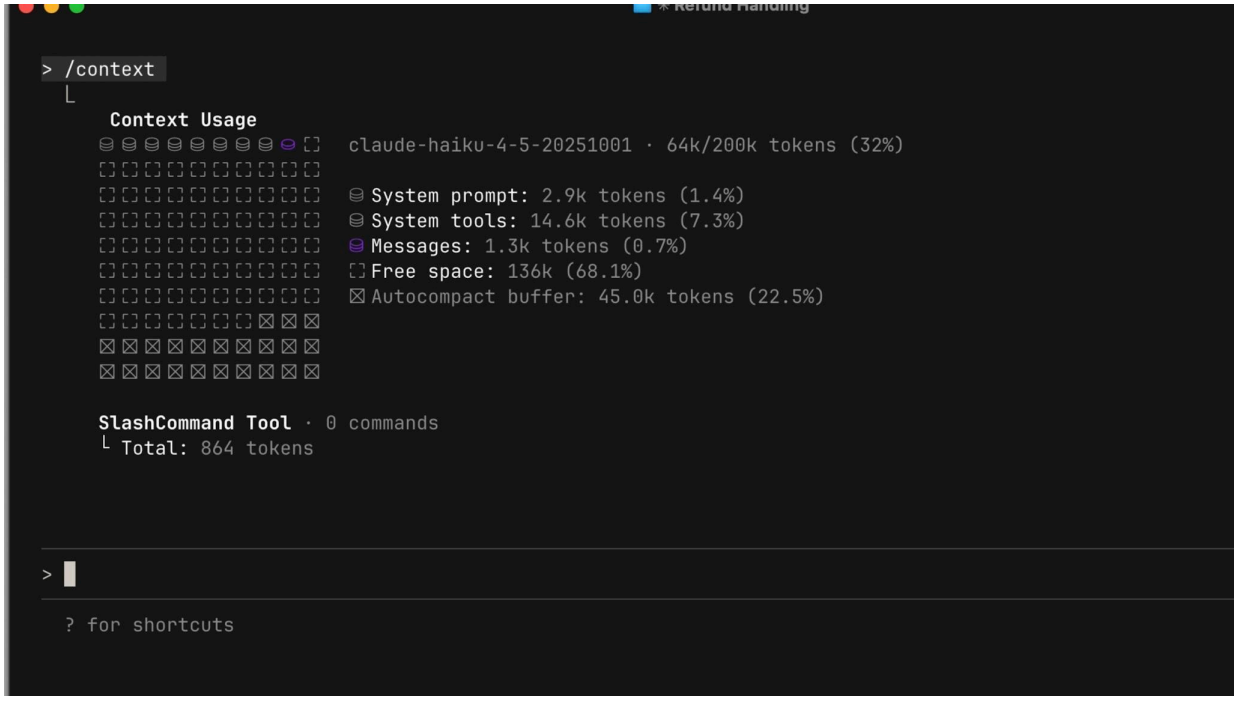
If you want to completely start from scratch with no memory of the previous session, run `/clear`. This removes everything.



/clear wipes the session entirely so you can begin again with a clean slate.

`/context` - see the breakdown

To check the state of your context, run `/context`. You'll get a high-level overview of your context size, the categories taking up the most space, and a visual graphic showing the breakdown.

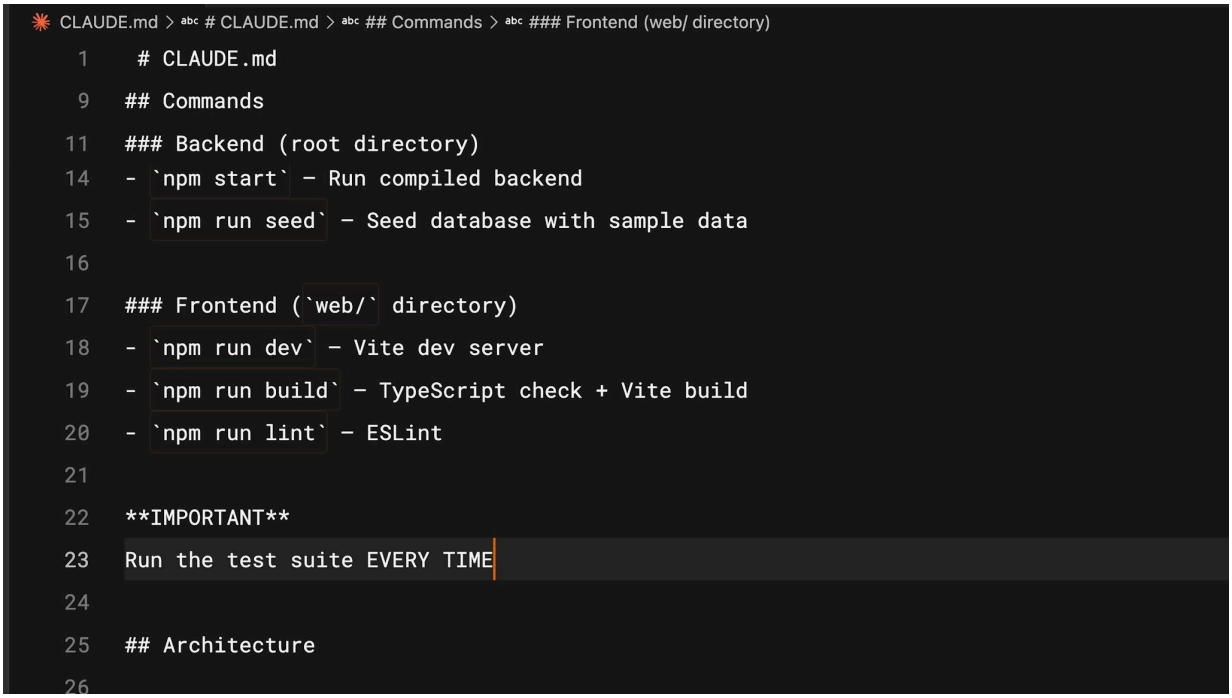


/context shows your usage, the biggest space consumers, and a visual breakdown.

04 When to Use Which

A general rule of thumb:

Reach for	When
`/compact`	You're working on a specific feature and running up against the context limit but need to continue. Keep the context relevant to what you're building.
`/clear`	You're starting a new feature and don't want the previous conversation to introduce bias into something new.



Put anything Claude should remember across sessions into your CLAUDE.md file.

PERSIST MEMORY WITH CLAUDE.MD

For things you want Claude to remember across sessions, put them in your `CLAUDE.md` file - so it doesn't have to rediscover them from scratch every time.

05 Tips for Saving Context Space

A few habits go a long way:

- **Be specific.** A vague prompt might seem smaller, but it actually costs more context in the long run. Without clear instructions, Claude is forced to explore your codebase and do its own reasoning - which takes far more space than a detailed prompt would.
- **Manage your MCP servers.** MCP servers load all of their available tools into context by default, even when you're not using them. If some are unrelated to the current project, consider turning them off. You can also try "Skills," which work similarly but don't load everything upfront.
- **Use subagents.** Subagents run in parallel with your main agent but have a completely separate context window. For tasks where you only need the answer - like "where are the

authentication endpoints located?” - a subagent does the work and returns just a summary, keeping your primary context clean.

06 Recap

Managing context within Claude Code is crucial. Use `/compact` to summarize long sessions and `/clear` to start fresh.

KEY TAKEAWAYS

- Use `/compact` to summarize long sessions; `/clear` to start a new one fresh.
- Be specific with your prompts - it saves Claude from costly exploration.
- Check what’s consuming your context with `/context`.
- Use subagents to delegate tasks where you only need the result.

> **Treat context like working memory - keep it lean, and Claude works sharper.**

Hands-On Workshop: Manage Context in Claude Code

Keep Claude's Working Memory Lean

/context · /compact · /clear

IN THIS DEMO

- Check how much context we're using and what's taking up space.
- Do some work to fill the context, then summarize it to keep going.
- Clear the context completely before starting an unrelated task.

ON THIS PAGE

01 What Context Is

Context is Claude's working memory for a session. Every prompt you send, every file Claude reads, and every command result takes up space in a **finite** context window. As it fills, Claude

can lose track of earlier details, and it will automatically compact (summarize) the conversation to free space - which can drop details along the way. Managing context deliberately keeps Claude sharp and avoids surprises.

The three commands:

Command	What it does
`/context`	Shows how full the window is and what is consuming the most space.
`/compact`	Summarizes the conversation so far and keeps the summary, freeing space while preserving memory of what you were doing.
`/clear`	Wipes the session entirely for a clean start. `CLAUDE.md` stays loaded.

02 Prerequisites

- Claude Code installed and running (`claude` is available in your terminal).
- The `context-management` folder from the workshop bundle (a small sample codebase). Open it in your terminal:

```
cd context-management
claude
```

Project Files - Link

03 Check Your Starting Context

Before doing anything, see the baseline:

```
/context
```

You will get an overview of how much of the window is in use and a breakdown by category - your `CLAUDE.md`, any connected MCP server tools, and the conversation so far. Note roughly how full it is.

04 Fill the Context with Real Work

Now do enough work to grow the context. Ask Claude to read and explain several files in the project:

```
Read the files in src/ and explain how a camera snapshot request flows
from the API route through to the device.
```

Claude reads `src/api/`, `src/integrations/`, `src/auth/`, and `src/stream/` and traces the flow. Every file it opens is now sitting in your context.

Check again:

```
/context
```

The usage has climbed, and the conversation/file-reads now take a visible slice. This is normal - it is the cost of the exploration you just did.

05 Compact to Keep Going

You are still working on the same area and want to continue, but free up space. Summarize the session:

```
/compact
```

Claude replaces the detailed history with a structured summary and keeps it in context. Run `/context` once more to confirm usage dropped. You can keep working on the snapshot flow - Claude still remembers what it found, just in condensed form.

COMPACTION LOSES DETAIL

If something matters for the long term, save it somewhere durable - see **Persist What Should Survive** below.

06 Clear Before an Unrelated Task

Now imagine you are switching to something completely different - say, the authentication module - and you do not want the snapshot exploration biasing the new work. Start fresh:

```
/clear
```

This wipes the session. Your `CLAUDE.md` remains loaded, so project conventions survive, but the snapshot conversation is gone. Run `/context` to see the window back near baseline.

07 Persist What Should Survive

Compaction and clearing both discard detail. Anything Claude should remember across sessions belongs in `CLAUDE.md`. For example:

```
Add a note to CLAUDE.md: snapshot requests must go through
src/api/snapshot.ts, which enforces TLS and validates the host.
```

From now on that fact is loaded every session and never has to be rediscovered.

08 Summary

- Context is finite working memory; every prompt, file, and result consumes it.
- `/context` shows usage and the biggest consumers - check it when things feel sluggish or before a big task.
- `/compact` summarizes and continues; `/clear` wipes for a fresh start.
- Use `/compact` to stay on the current feature near the limit; use `/clear` when moving to unrelated work.
- Persist anything durable in `CLAUDE.md` so it survives compaction and clearing.

09 Tips for Saving Context Space

- **Be specific.** A vague prompt looks smaller but costs more, because Claude has to explore and reason to fill the gaps. Clear instructions are cheaper overall.
- **Manage MCP servers.** Each connected server loads all of its tools into context, even unused ones. Turn off servers you do not need for the current task. Skills are a lighter alternative - they do not load everything up front.

- **Use subagents.** When you only need an answer (for example, “where is device authentication validated?”), delegate to a subagent. It works in its own context and returns just a summary, keeping your main window clean.

10 Troubleshooting

Symptom	Resolution
Context fills surprisingly fast	Run `/context` to find the consumer. Large file reads or many MCP tools are common causes.
Claude forgot something after compaction	It was not durable. Put it in `CLAUDE.md` and it will reload each session.
New task feels biased by old work	You compacted when you should have cleared. Use `/clear` when switching to unrelated work.

> **Inspect with /context, summarize with /compact, reset with /clear - and keep anything durable in CLAUDE.md.**